

Do Large Language Models Understand High-Level Message Sequence Charts?

Mohammad Reza Mousavi¹[0000–0002–4869–6794]

Department of Informatics, King’s College London mohammad.mousavi@kcl.ac.uk

Abstract. Large Language Models (LLMs) are being widely employed to automate tasks across the software development lifecycle. It is, however, unclear whether these tasks are performed consistently with respect to the semantics of the objects being handled. This question is particularly under-researched concerning the high-level design and specification tasks, at the architectural level.

In this paper, we address this question for High-Level Message Sequence Charts (HMSCs). These are visual models with a rigorous formal semantics that have been used for various purposes, including as a foundation for Sequence Diagrams in the Unified Modelling Language (UML). We examine whether LLMs “understand” the semantics of HMSCs by examining three LLMs (Gemini-3, GPT-5.4, and Qwen-3.6-Plus) on how they perform 129 semantic tasks ranging from querying basic semantic constructs in HMSCs (i.e., events and their ordering) to semantic-preserving abstractions and compositions, and calculating the set of traces and a trace-equivalent labelled transition system. The results show that while LLMs seem to understand the basic semantics of MSCs (~88% accuracy), they struggle with semantic reasoning in tasks involving abstraction and composition (~36% accuracy) and traces and LTSs (~42% accuracy). In particular, all three LLMs struggle with the notions of co-region and explicit causal dependencies and never employed them in semantic-preserving transformations.

Keywords: Large Language Models · Architectural Design · Design Comprehension · Semantics · High-Level Message Sequence Charts · Unified Modelling Language (UML).

To Sjouke Mauw and Michel Reniers,
for their contributions to
the formal semantics of
Message Sequence Charts; and
To Michel A. Reniers,
for teaching me formal semantics....

1 Introduction

1.1 Motivation

High-level Message Sequence Charts (HMSCs) [5] are among the pioneering models for software and system specification, which are furnished with a formal semantics. They have been widely used in practice, standardised, and have also been integrated into the Unified Modelling Language [1] in the form of sequence diagrams.

1.2 Problem Definition

Understanding architectural specification is among the challenging usecases for Large Language Models (LLMs). By “understanding” we do not mean to anthropomorphise LLMs [11]; rather, we are referring to their observable behaviour: demonstrating outputs that are consistent with a sound model of the semantics of HMSCs, e.g., in answering queries and proposing manipulations and generating models. It is unclear if LLMs respect the semantics of architectural specification languages, even with respect to their most fundamental semantics construct. In this study, our aim was to quantify the understanding of a basic understanding of HMSCs by first identifying their most fundamental semantic concepts and then measuring LLM’s understanding by posing queries that pertain to explanation, generation, and manipulation of HMSCs for each and every semantic concept.

1.3 Approach

To this end, we 1) identify the key semantic concepts in the formal semantics of HMSCs, 2) formulate and carry out tasks that examine their semantic understanding, and 3) analyse the performance of LLMs and provide a quantification of the understanding of various semantic concepts.

Key semantic concepts. The formal semantic concepts are drawn from the formalisation of HMSCs, based on the ITU Recommendation Z.120 Annex B: Algebraic Semantics of Message Sequence Charts [5]. The identified semantics concepts are: causal ordering of events, abstraction, composition, traces, and (trace-equivalent) labelled transition systems. We expand on each of these concepts and identify patterns that exemplify them in the subsequent explanation, generation, and manipulation tasks.

1.4 Results

Our results indicate that the three LLMs evaluated in our paper, all have a precise model of the basic semantics constructs of (H)MSCs: they are all accurate in extracting the events and event orderings from (H)MSCs ($\sim 88\%$ accuracy). However, they struggle with abstraction, composition ($\sim 36\%$ accuracy), and higher-level reasoning about behavioural semantics ($\sim 42\%$ accuracy). They all

seem to lack a sound notion of co-region (a region with no ordering of events) and explicit causal orders among events. For example, they never use co-regions when composing (H)MSCs horizontally (in parallel), leading to introducing fictitious orders in such compositions. Also, when abstracting from events, they cannot reinstate the ordering implied by the abstracted events, leading to fewer orders than the original (H)MSC. Another interesting observation is that in the few cases of successful higher-level reasoning, they generate Python code for the corresponding reasoning task and execute it to obtain a sound result. This indicates that potentially reliable results can be obtained by interfacing LLMs with other tools and encoding the reasoning tasks into the input of such external tools.

1.5 Organisation

The remainder of this paper is organised as follows. We review the related work to position our work with respect to the literature on understanding code and architectural diagrams in the context of Large Language Models in Section 2. Then we recap some basic concepts regarding the syntax and semantics of (H)MSCs in Section 3. In the same section, we identify the key semantic concepts that we would like to examine in our experiments. This leads to a formulation of our research questions and the design of experiments to answer them in Section 4. We report the results of our experiments and answer the formulated research questions in Section 5. We reflect on these answers and on the threats to the validity and the generalisability of the results in Section 6. Finally, we conclude the paper and present the directions of future work in Section 7.

2 Related Work

In this section, we review the related lines work on measuring semantic understanding in LLMs for code and for architectural specifications.

Understanding Program Semantics. Although understanding code semantics may deal with more fine-grained semantic issues, many of the semantic concepts and tasks examined in this paper are relevant for code-level (operational) semantics, particular for reasoning about concurrency.

Ma et al. [7] analyse the LLM capabilities in code understanding at three levels: syntax, static semantics, and dynamic semantics. For understanding dynamic semantics, they report an accuracy of ca. 60% using zero-shot (and slightly lower using few-shot) prompting. Laneve et al. [6] analyse program semantic understanding among seven different LLMs. They do so by generating tasks about program equivalence and analysing the correctness of the outcome. They report a mean accuracy of 59% in zero-shot prompting and 71% after providing contextual information about the perturbation in the programs. They conclude that there is a significant gap in the semantic reasoning capabilities of the analysed LLMs. Similarly, Wei et al. [16] analyse the capabilities of 19 LLMs to reason

about the programme equivalence and report a mean precision of 64%. Hooda et al. [4] use counter-factual analysis to analyse LLM program understanding: they use four different Program Concept Predicates to characterise four classes of semantic equivalence in programs. They use these concepts to evaluate the performance of LLMs in code completion and program repair tasks. Their analysis shows significant gaps in understanding program semantics that influence downstream task precision by up to 33%. All of these approaches address the issue of semantic understanding, and point out significant gaps in understanding code semantics. Our methodology differs from all of these approaches, since they use a typical downstream task such as equivalence checking or code completion to analyse semantic understanding. However, in our approach, we directly test the semantic concepts such as ordering and traces in our queries, which give us a more direct access to semantic understanding metrics.

Nikiema et al. [8] used obfuscation to measure semantic understanding; they used the accuracy of generating explanation and deobfuscation as proxies for understanding and showed that code obfuscation leads to significant decline in the accuracy of both tasks.

In another line of enquiry, Spiess et al. [15] study the robustness of code semantic understanding. The use input perturbations to check the robustness of output prediction tasks as a proxy for understanding dynamic semantics. Their results, in line with all the previous line of research, show lack of robustness in the understanding of dynamic semantics.

In a different line of work, North et al. [9], analyse whether there is a common semantic understanding in LLMs across different programming languages. Their initial results suggest that such a common semantic understanding does exist. While this problem is orthogonal to the problem studied in this paper, further insights about the common semantic understanding across different languages can prove essential; it may enable exploring the boundaries of semantic understanding in LLMs and extending and augmenting it.

Table 1 provides an overview of the comparison of related work in code semantic understanding; although our paper is not directly about programming semantics, we do think the approach and results could be useful in this domain as identified in the last row. In particular, our methodology in formulating semantic tasks that directly examine the basic semantic concepts is complementary to all of the reviewed pieces of literature in Table 1 and can be employed to devise novel studies that go beyond the available research.

Architectural Understanding. Schmid et al. [12] perform a systematic review of the use of LLMs in architectural tasks in software engineering. The results indicate that there is very limited research on the suitability of LLMs for architectural understanding. Below we review some of the recent literature addressing this research gap.

For example, Soliman et al. [13, 14] study the availability and accuracy of architectural knowledge in LLMs. They evaluate three types of architectural questions and report a precision of below 20% and a recall of 20%-40% among seven LLMs. Corroborating this report, Zhou et al. [17] report a precision of ca.

Table 1. Comparison with work on code semantic understanding.

Work	Type of study	Metrics used
Ma et al. [7]	Code syntax/semantics	Accuracy
Laneve et al. [6]	Program equivalence	Accuracy
Wei et al. [16]	Program equivalence	Precision
Hooda et al. [4]	Counterfactual predicates	Downstream precision
Nikiema et al. [8]	Code obfuscation	Task accuracy
Spiess et al. [15]	Execution perturbations	Robustness/accuracy
North et al. [9]	Cross-language semantics	Comparative performance
Our paper	HMSC semantic bench- mark	Stratified accuracy

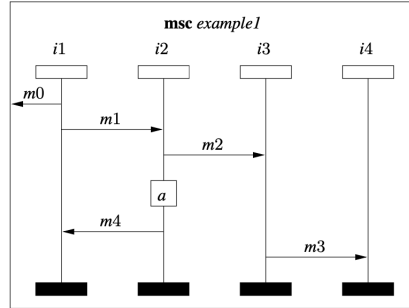
27% and recall of ca. 66% regarding explanations of design rationale. Elberzhager et al. [3] use case studies to evaluate the consistency and accuracy of LLM-supported evaluation of architecture documents. The provided analysis is brief and is confined to two case studies, but they suggest high consistency and low precision. Gurerra and Ernst analyse the performance of LLMs with respect to architectural tasks; they categorise the tasks into five levels: remembering, understanding, applying, analyzing, evaluating, and creating. They use a case study for their experiment and they report surprisingly high precision scores (called grades) across all tasks, with slightly lower scores on the initial (lower-level) tasks. We expect these high scores are motivated by the generic nature of the analysed case study and the fact that LLMs have potentially been trained on similar architectures.

Ouyang et al. [10] study architectural understanding from a multimodal perspective by evaluating Vision-Language Models on software architecture diagrams. They introduce SADU, a benchmark comprising 154 behavioural, structural, and entity-relationship diagrams with 2,431 question-answering tasks targeting counting and retrieval reasoning over architectural elements. Their evaluation of 11 state-of-the-art VLMs shows that diagram-based architectural understanding remains challenging: the best model reaches only 70.18% accuracy, with performance degrading on more complex diagrams and on visually difficult relation structures such as overlapping or non-standard arrows. The current paper complements the study by Ouyang et al. [10] in exploring the boundaries of LLM analysis regarding deeper semantical concepts.

Table 2 summarises the comparison of the related work on architectural understanding. It appears from the existing literature that we lack a fundamental understanding LLMs capabilities in architectural understanding. In particular, we do not seem to have a precise characterisation of the boundaries of correct architectural reasoning in LLMs. The goal of the present paper is to take the first step in this direction for the specific case of Message Sequence Charts.

Table 2. Comparison with work on architectural understanding.

Work	Type of study	Metrics used	Gap filled by our paper
Schmid et al. [12]	Systematic review	Literature synthesis	No semantic benchmark
Soliman et al. [13, 14]	Architecture knowledge QA	Precision, recall	General knowledge only
Zhou et al. [17]	Design-rationale generation	Precision, recall	Rationale, not semantics
Elberzhager et al. [3]	Document case studies	Consistency, precision	Document-level evaluation
Guerra and Ernst	Architecture task taxonomy	Grades/precision	Broad case-study tasks
Ouyang et al. [10]	Architecture VQA	Accuracy	Visual grounding only
Our paper	HMSC benchmark	semantic Stratified racy	accu- Direct formal semantics

**Fig. 1.** An MSC labelled “example1”, with four instances, labelled “i1” to “i4”, five messages labelled “M0” to “M4”, and an internal action labelled “a”.

3 High-Level Message Sequence Charts

3.1 Syntax

A message sequence chart (MSC) is given by a labelled frame (a bounding box) comprising a number of instances, represented by vertical lines labelled by the instance names. Intuitively, they can be thought of as objects in a software system, or nodes in a distributed system. They send and receive messages, denoted by labelled arrows across instances. They may also perform internal actions, denoted by squares placed on vertical lines. Message can, for example, represent method calls in software or packet send and receive actions on a network. In addition to messages sent and received between instances, they can also be sent and received to and from the environment denoted by arrows starting from and ending at the frame, respectively. An example of an MSC is depicted in Figure 1. This is the first subject system used in our experiments. Below we explain the syntactic elements used in this example.

Example 1 (MSC: Syntax). Figure 1 depicts an MSC labelled “example1”. It comprises four instances, labelled “i1” to “i4”; messages “M1” to “M4” are sent and received between instances, while message “M0” is sent to the environment. Internal action a takes place within instance “i2”. Identifying the events in this example was the first task defined in our experiments.

HMSCs are the result of composing MSCs using choice operator, depicted by a branch, vertically (sequentially), denoted by sequencing, and horizontally (in parallel) denoted by juxtaposing the MSCs. Choice and sequential composition can be combined to represent iteration (loop). Figures 2 and 2 depict and HMSC and its components MSCs, respectively.

Example 2 (HMSC: Syntax). Figure 2 depicts an HMSC which starts off with the behavior specified by the MSC labelled “Connection request”, vertically (sequentially) composed by a choice between “Connection confirm” and “Time-out”. The MSCs referenced in this HMSC are depicted in Figure 3.

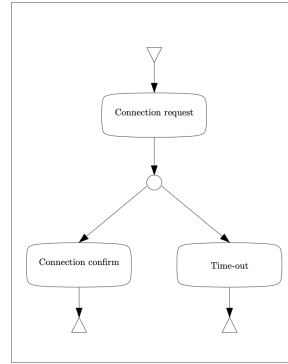


Fig. 2. An HMSC labelled comprising a vertical composition of “Connection request” with a choice between “Connection confirm” and “Time-out”.

3.2 Semantics

The identified key semantic concepts are listed below:

1. Causal ordering: the most important semantic concept in Basic Message Sequence Charts (BMSCs, the building blocks of HMSCs) is the causal ordering of events; they comprise the following types of orders:
 - (a) Send and receive order: a message cannot be received before it is sent;
 - (b) Vertical order: events in the same instance and on the same vertical line are totally ordered from top to bottom;

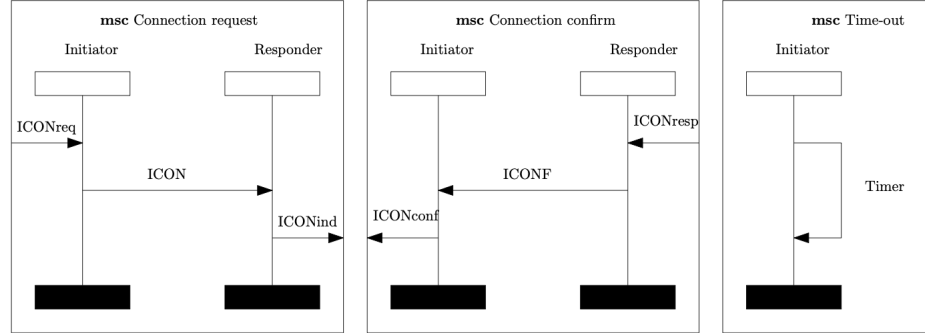


Fig. 3. The MSCs referenced in the HMSC of Figure 2.

- (c) Transitive closure: causal ordering of events is a transitive relation.
- 2. Abstraction: an event can be abstracted from a BMSC, leading to an equivalent BMSC with fewer events but the same causal ordering among the remaining events; moreover, entities can be merged to create BMSCs with fewer entities.
- 3. Composition: BMSCs can be composed using various composition operators; to confine the scope of the study, we focus on the two most fundamental composition operators, namely, horizontal and vertical composition.

Below we explain the semantic concepts relevant for the (H)MSCs depicted in Figures 1, 2, and 3.

Example 3. Consider the MSC depicted in Figure 1. Sending “M0” happens before sending “M1” because they are ordered by the vertical line of instance “i1”. Sending “M1” happens before receiving “M1” due to the causal order between send and receive. Sending “M1” happens before receiving “M3” due to the causal order in the sends and receives for “M1”, “M2”, and “M3”, the vertical order between receiving “M1” and sending “M2” and receiving “M2” and sending “M3”, and finally the transitive closure of all these orders. In fact, this is the second task given to the three LLMs to examine their semantic understanding.

Example 4. Consider the MSC depicted in Figure 1. Sending “M0” happens before sending “M1” because they are ordered by the vertical line of instance “i1”. Sending “M1” happens before receiving “M1” due to the causal order between send and receive. Sending “M1” happens before receiving “M3” due to the causal order in the sends and receives for “M1”, “M2”, and “M3”, the vertical order between receiving “M1” and sending “M2” and receiving “M2” and sending “M3”, and finally the transitive closure of all these orders. In fact, this is the second task given to the three LLMs to examine their semantic understanding.

Additionally, the event orders specified above gives rise to a set of traces, a totally-ordered sequence of events respecting the orders. Moreover, the semantics of the MSC can be presented by a labelled transition system capturing all the events and event ordering specified by the semantics (modulo trace equivalence).

4 Methodology

This section describes the methodology used to evaluate the extent to which contemporary Large Language Models (LLMs) can understand and reason about Message Sequence Charts (MSCs) and High-Level Message Sequence Charts (HMSCs). The study is organised around a controlled benchmark of subject models, a set of semantic reasoning and transformation tasks, and a comparative evaluation across multiple LLMs.

4.1 Research Questions

The study is guided by the following research questions:

RQ1: To what extent can LLMs identify and reason about the basic semantic elements of MSCs?

This question evaluates whether LLMs can recognise the foundational building blocks of the semantics of MSCs, including events (send, receive and internal actions), and causal ordering relations (induced by send and receive, by the vertical order, and by the transitive closure of the former two).

RQ2: How well do LLMs perform semantic transformations over MSCs?

This question concerns operations that require more than having a notion of the basic semantic constructs. The evaluation includes transformations such as abstraction from events and merging of instances, where the resulting MSC must preserve the relevant behavioural semantics of the original chart.

RQ3: Can LLMs reason about the behavioural semantics of MSCs?

This question assesses whether LLMs can derive behavioural representations from MSCs. In particular, it considers whether models can identify the set of traces represented by an MSC and generate a labelled transition system with equivalent trace semantics.

RQ4: How does LLM performance vary with model complexity and task complexity?

This question investigates whether the accuracy of LLMs depends on the complexity of the subject MSC/HMSC and on the complexity of the task being performed. The evaluation distinguishes between simple and complex models, as well as between simple tasks, such as event identification, and complex tasks, such as abstraction, composition, interleaving, and trace generation.

RQ5: Which MSC concepts constitute recurring “pain patterns” for LLMs?

This question aims to identify classes of MSC reasoning problems that systematically challenge LLMs. The preliminary results suggest that abstraction, composition, concurrency, and interleaving are particularly difficult for the evaluated models.

4.2 Subject Systems

The subject systems consist of nine MSC and HMSC models taken from Software Specification lecture notes. These models were selected because they provide a controlled and pedagogically validated corpus of examples. Their use offers two main advantages. First, there is a high degree of confidence that the models are well formed. Second, solutions to the corresponding exercises are available, allowing the generated LLM responses to be compared against reference answers.

The benchmark covers both basic MSCs and higher-level MSC constructs. The basic concepts include events, send and receive actions, internal events, causal ordering, vertical ordering along instances, transitive closure, abstraction from events, merging of instances, and coregions. The higher-level concepts include horizontal and vertical composition, HMSC notation, and composition operators.

A limitation of this choice of subject systems is that the models originate from lecture notes, and some LLMs may have been exposed during training to unofficial copies of these materials. This potential data contamination risk is therefore considered a threat to validity.

4.3 Evaluated Models

The evaluation compares the performance of three contemporary LLMs:

- Gemini-3-Pro Thinking;
- GPT-5.4 Think Deeper;
- Qwen-3.6-Plus Thinking.

These models were selected as representative examples of state-of-the-art generative AI systems. The purpose of comparing multiple LLMs is to determine whether the observed strengths and weaknesses are model-specific or whether they reflect broader limitations in current LLM-based reasoning about MSCs and HMSCs.

4.4 Evaluation Tasks

The evaluation tasks are derived from the semantic concepts underlying MSCs and HMSCs. Each task is expressed as a natural-language prompt and accompanied by a diagrammatic MSC or HMSC input where appropriate.

Event Identification The first class of tasks asks the model to identify the events present in a given MSC. This includes distinguishing between send events, receive events, and internal events. These tasks evaluate whether the model can correctly recognise the basic syntactic and semantic units of an MSC.

Causal Ordering The second class of tasks evaluates whether the model can determine ordering relations between events. For example, a model may be asked to determine the ordering between sending two messages, or between receiving one message and sending another. Correct answers require the model to account for send–receive order, vertical ordering along instances, and the transitive closure of these relations.

Abstraction from Events The third class of tasks concerns abstraction. In these tasks, the model is asked to generate or reason about an MSC in which a given message or event has been abstracted away. Such tasks test whether the model can preserve the correct observable behaviour after hiding or removing specified events. They also require the model to update causal dependencies appropriately after abstraction. Also in the same category, we ask the model to merge two instances in an MSC and produce the resulting chart. This evaluates whether the model can correctly combine lifelines while preserving the event ordering and communication dependencies induced by the original MSC.

Trace Identification The fifth class of tasks requires the model to compute the set of traces represented by an MSC. These tasks assess the model’s ability to reason about concurrency and interleaving. Since MSCs often denote partial orders rather than single total orders, identifying all possible traces requires systematic reasoning about the linear extensions of the underlying causal ordering.

Generation of Labelled Transition Systems The final class of tasks asks the model to generate a labelled transition system with the same traces as the given MSC. These tasks require translation from a diagrammatic specification to an operational behavioural representation. They therefore evaluate whether the model can move beyond recognition of diagram elements and construct an equivalent formal model.

4.5 Experimental Procedure

The experimental procedure consists of the following stages.

Benchmark construction. A benchmark of nine MSC/HMSC subject models is constructed from Software Specification lecture notes. The selected models cover a range of semantic concepts, from basic event identification and causal ordering to more advanced operations involving abstraction, merging, composition, concurrency, and trace semantics.

Prompt design. For each subject model, task-specific natural-language prompts are created. The prompts ask the LLM to perform a particular semantic operation, such as identifying events, determining an ordering relation, abstracting a message, merging instances, computing traces, or generating a labelled transition system. The prompts are designed to evaluate both local diagram understanding and global semantic reasoning.

LLM execution. Each prompt is submitted to each of the three evaluated LLMs. The models produce textual, symbolic, or diagrammatic responses, depending on the nature of the task. All generated responses are recorded for subsequent comparison with reference solutions.

Reference-based assessment. The generated outputs are assessed against the available reference solutions. A response is considered correct if it preserves the intended MSC semantics. For event-identification tasks, this requires identifying the correct set of events. For causal-ordering tasks, it requires deriving the correct ordering relation. For abstraction and merging tasks, it requires producing a semantically valid transformed MSC. For trace and labelled-transition-system tasks, it requires preserving the trace semantics of the original MSC.

Aggregation of results. Correctness judgements are aggregated to obtain overall accuracy scores. The results are further stratified by model complexity and task complexity. This allows the evaluation to distinguish between performance on simple and complex MSCs, and between performance on basic and advanced reasoning tasks.

Error analysis. Finally, incorrect responses are analysed to identify recurring failure modes. Particular attention is paid to failures involving abstraction, composition, concurrency, and interleaving, since these concepts appear to be substantially more challenging for the evaluated models than basic event identification and causal ordering.

4.6 Evaluation Metrics

The primary evaluation metric is accuracy, defined as the proportion of tasks for which the LLM produces a correct answer according to the reference solution. The overall accuracy reported in the study is 52%. However, the results show substantial variation across task and model complexity. The evaluated models achieve 66% accuracy on simple models, compared with 33% on complex models. Similarly, they achieve 94% accuracy on simple tasks, compared with 25% on complex tasks.

These results indicate that current LLMs are capable of handling some foundational aspects of MSCs, especially event identification and basic causal ordering. However, their performance deteriorates significantly on tasks requiring abstraction, composition, concurrency reasoning, interleaving analysis, and the construction of equivalent behavioural models.

4.7 Pipeline Implementation

The evaluation pipeline implements the above procedure as a sequence of benchmark preparation, prompt execution, response collection, semantic assessment, and result aggregation steps. First, the MSC/HMSC diagrams and their corresponding reference solutions are collected from the lecture-note corpus. Second,

for each diagram, a set of prompts is instantiated according to the task categories described above. Third, each prompt is submitted independently to each evaluated LLM. Fourth, the generated responses are compared with the reference solutions using task-specific correctness criteria. Finally, the resulting correctness judgements are aggregated into overall and stratified accuracy measures.

The pipeline is intentionally designed around semantic rather than purely syntactic correctness. For example, in abstraction tasks, a response is not judged solely by whether it removes the requested message, but by whether the resulting MSC preserves the correct observable behaviour. Similarly, in trace-generation tasks, a response is judged by whether it captures all and only the traces permitted by the causal structure of the original MSC. This semantic orientation is essential because the aim of the study is not merely to test visual recognition of MSC diagrams, but to evaluate whether LLMs can reason about their formal behavioural meaning.

4.8 Summary

In summary, the methodology evaluates three LLMs on a controlled benchmark of nine MSC/HMSC subject models. The benchmark covers both basic and advanced semantic concepts, including event identification, causal ordering, abstraction, merging, trace identification, and labelled-transition-system generation. The evaluation compares model outputs against reference solutions and analyses performance across task and model complexity. The preliminary results suggest that LLMs can handle basic MSC concepts, but struggle with abstraction, composition, concurrency, and interleaving. These findings motivate future work on formulating and testing concrete hypotheses about recurring LLM “pain patterns” in formal diagram understanding.

5 Results

5.1 Analysis Results

Coverage check	Check	Result
	Total judged tasks	129
	Models	3
	Judged tasks per model	43
	Examples per model	9
	Unknown/unclassifiable judgments	0

Discrepancy report: No discrepancy was found in the number or type of tasks across models after normalization. All models were tested on the same number of tasks and the same task coverage.

Accuracy by LLM and task type

Table 3. Summary of performance per task type for GPT-5.4-Think Deeper

Task type	Correct	Incorrect	Total	Accuracy
Identifying events	3	1	4	75.0%
Identifying event orders	5	2	7	71.4%
Abstraction	2	8	10	20.0%
Traces	3	1	4	75.0%
LTS	2	2	4	50.0%
Composition	5	9	14	35.7%
Overall	20	23	43	46.5%

Table 4. Summary of performance per task type for Gemini 3 - Thinking

Task type	Correct	Incorrect	Total	Accuracy
Identifying events	4	0	4	100.0%
Identifying event orders	7	0	7	100.0%
Abstraction	1	9	10	10.0%
Traces	2	2	4	50.0%
LTS	0	4	4	0.0%
Composition	10	4	14	71.4%
Overall	24	19	43	55.8%

Table 5. Summary of performance per task type for Qwen 3.6 Plus - Thinking

Task type	Correct	Incorrect	Total	Accuracy
Identifying events	4	0	4	100.0%
Identifying event orders	6	1	7	85.7%
Abstraction	3	7	10	30.0%
Traces	2	2	4	50.0%
LTS	1	3	4	25.0%
Composition	5	9	14	35.7%
Overall	21	22	43	48.8%

Table 6. Summary of total accuracy per task type accross the three LLMs.

Task type/Accuracy	GPT	Gemini	Qwen
Identifying events	75.0%	100.0%	100.0%
Identifying event orders	71.4%	100.0%	85.7%
Abstraction	20.0%	10.0%	30.0%
Traces	75.0%	50.0%	50.0%
LTS	50.0%	0.0%	25.0%
Composition	35.7%	71.4%	35.7%

5.2 Summary performance by task type

5.3 RQ1: To what extent can LLMs identify and reason about the basic semantic elements of MSCs?

LLMs performed strongly on the basic semantic elements of MSCs, especially when the task required identifying events or determining event ordering. Across all models, the combined accuracy for these basic semantic tasks was 87.9%.

Table 7. Performance on basic MSC semantic tasks.

Basic MSC task	Correct Total Accuracy		
Identifying events	11	12	91.7%
Identifying event orders	18	21	85.7%
Combined basic semantic tasks	29	33	87.9%

This suggests that all three LLMs can usually recover the surface-level semantic structure of MSCs. They are particularly reliable at recognising individual send and receive events and inferring straightforward ordering constraints. However, performance is not perfect. GPT-5.4-Think Deeper was weaker than the other two models on these basic tasks, scoring 75.0% on identifying events and 71.4% on identifying event orders, while Gemini achieved 100.0% on both and Qwen achieved 100.0% and 85.7%, respectively.

Answer

LLMs are generally competent at identifying and reasoning about basic MSC semantic elements. Basic event recognition and event ordering are the strongest areas in the dataset.

5.4 RQ2: How well do LLMs perform semantic transformations over MSCs?

Semantic transformations were substantially harder than basic semantic recognition. The two transformation-oriented categories were abstraction and composition. Across all models, the combined accuracy for transformation tasks was only 36.1%.

Table 8. Performance on MSC semantic transformation tasks.

Transformation task	Correct	Total	Accuracy
Abstraction	6	30	20.0%
Composition	20	42	47.6%
Combined transformation tasks	26	72	36.1%

Abstraction was the weakest category overall. All three models struggled with it. Gemini achieved only 10.0%, GPT-5.4-Think Deeper achieved 20.0%, and Qwen achieved 30.0%.

Table 9. Model-level performance on abstraction and composition.

Model	Abstraction accuracy	Composition accuracy
GPT-5.4-Think Deeper	20.0%	35.7%
Gemini 3 - Thinking	10.0%	71.4%
Qwen 3.6 Plus - Thinking	30.0%	35.7%

Composition showed more model-dependent variation. Gemini performed relatively well at 71.4%, whereas GPT-5.4-Think Deeper and Qwen both achieved only 35.7%. This indicates that composition is not uniformly beyond LLM capability, but model-specific reasoning behaviour matters greatly. Abstraction, by contrast, appears to be a systematic weakness across all models, likely because it requires preserving semantic equivalence while removing or hiding details.

Answer

LLMs perform poorly on MSC semantic transformations overall. Abstraction is a severe weakness, while composition is possible for some models but inconsistent.

5.5 RQ3: Can LLMs reason about the behavioural semantics of MSCs?

Behavioural semantics were represented by traces and labelled transition system (LTS) tasks. Across all models, behavioural-semantics accuracy was 41.7%.

Table 10. Performance on behavioural semantics tasks.

Behavioural task	Correct	Total	Accuracy
Traces	7	12	58.3%
LTS	3	12	25.0%
Combined behavioural semantics tasks	10	24	41.7%

The models showed moderate ability on traces but poor performance on LTS reasoning.

Table 11. Model-level performance on traces and LTS.

Model	Traces accuracy	LTS accuracy
GPT-5.4-Think Deeper	75.0%	50.0%
Gemini 3 - Thinking	50.0%	0.0%
Qwen 3.6 Plus - Thinking	50.0%	25.0%

Trace reasoning appears easier than LTS construction or interpretation. This is plausible because traces can often be checked as linear sequences, whereas LTS reasoning requires representing branching behaviour, states, transitions, and possibly nondeterminism.

Answer

LLMs can reason about traces to a limited or moderate extent, but they struggle substantially with LTS-based behavioural semantics. LTS is one of the clearest failure areas.

5.6 RQ4: How does LLM performance vary with model complexity and task complexity?

Overall model performance was fairly close, with Gemini achieving the highest aggregate accuracy.

Table 12. Overall model performance.

Model	Correct	Total	Overall accuracy
Gemini 3 - Thinking	24	43	55.8%
Qwen 3.6 Plus - Thinking	21	43	48.8%
GPT-5.4-Think Deeper	20	43	46.5%

The results do not show a simple monotonic relationship in which the apparently more advanced or deeper-thinking model is always better. Gemini had the best overall score, but its performance was uneven: it was excellent on event identification, event ordering, and composition, but failed all LTS tasks. GPT-5.4-Think Deeper was weaker overall, but strongest on traces and better than the others on LTS. Qwen was the best on abstraction, although abstraction accuracy remained low.

Task complexity had a clearer effect than model identity.

Table 13. Accuracy by broad task-complexity group.

Task group	Accuracy
Basic semantic tasks: events and event orders	87.9%
Behavioural semantics: traces and LTS	41.7%
Transformations: abstraction and composition	36.1%

Example-level results also show strong variation. Examples 1–3 and 9 combined multiple task types, while Examples 4–8 focused on composition. Example 8 was failed by all models, and Example 6 was also difficult, particularly for GPT-5.4-Think Deeper and Qwen.

Table 14. Aggregate performance by example.

Example	Aggregate correct	Aggregate total	Accuracy
1	8	21	38.1%
2	14	21	66.7%
3	9	21	42.9%
4	8	12	66.7%
5	7	12	58.3%
6	3	12	25.0%
7	2	3	66.7%
8	0	3	0.0%
9	14	24	58.3%

Answer

Task complexity explains performance better than model choice. LLMs are strongest on basic recognition, weaker on behavioural semantics, and weakest on semantic transformations, especially abstraction. Model differences exist, but no model dominates across all task types.

5.7 RQ5: Which MSC concepts constitute recurring pain patterns for LLMs?

The clearest recurring pain patterns are those with below-majority success across the full dataset.

Table 15. Recurring MSC pain patterns.

Pain pattern	Accuracy Interpretation
Abstraction	20.0% Severe recurring failure across all models.
LTS	25.0% Severe difficulty with state-transition behavioural semantics.
Composition	47.6% Borderline but still mostly incorrect overall.
Complex composition examples, especially Example 8	0.0% Failed by all models.

The strongest pain pattern is abstraction. Every model failed most abstraction tasks: Gemini scored 10.0%, GPT-5.4-Think Deeper scored 20.0%, and Qwen scored 30.0%. This suggests that abstraction is not just a model-specific weakness but a general limitation.

The second major pain pattern is LTS reasoning. Gemini failed all LTS tasks, Qwen solved only one out of four, and GPT-5.4-Think Deeper solved two out of four. This indicates that LLMs often struggle to convert MSC semantics into an explicit behavioural model.

The third pain pattern is composition, especially in more complex examples. Although Gemini performed well on composition overall, GPT-5.4-Think Deeper and Qwen did not. Example 8 was failed by all models, making it a strong candidate for a structurally difficult MSC composition case.

Answer

The recurring MSC pain patterns are abstraction, LTS reasoning, and difficult composition cases. These are the concepts that LLMs most often fail to handle correctly.

6 Reflections

6.1 Threats to Validity

Several threats to validity should be considered. First, the use of lecture-note examples provides a reliable source of well-formed models and reference solutions, but it also introduces the possibility that some models may have encountered unofficial copies of the material during training. Second, the benchmark contains nine subject models, which is sufficient for an exploratory study but should be expanded in future work to support stronger statistical conclusions. Third, correctness assessment may involve semantic judgement, particularly for diagram transformations and labelled transition systems; therefore, future versions of the benchmark should aim to define more precise and automatable checking procedures. Finally, the evaluation focuses on a selected set of LLMs, and results may change as new models and reasoning interfaces become available.

6.2 Environmental Sustainability

The amount of CO₂ emitted while performing this research has been a major unaddressed concern. Our past research [2] demonstrated that LLMs are likely to have an order of magnitude more of CO₂ emission compared to manual for comparable tasks compared to manual development. We did attempt to analyse how much CO₂ emission the experiments and the LLM assistance in the analysis of the paper results have costed, but the present tools are very limited, particularly in estimating the amount of CO₂ emission arising from prior training (and amortising it on inference queries). This, as well as other environmental and economic sustainability impacts, remains a major concern for our future research.

6.3 Declaration of the Use of AI

We have used AI (GPT 5.5 - ThinkingDeeper) to formulate the research questions, validate the consistency of queries and judgments across all experiments, analyse and summarise the judgments, and summarise the answers to the research questions. The prompts used for the experiments and their validations are publicly available in the replication package. The formatting and the landing page of the replication package was also generated using AI (GPT 5.5 - ThinkingDeeper). The introduction, related work, basic (semantic) concepts, and formulation of tasks and prompts were done manually by the author. Tables 1 and 2 summarising the related work were developed using AI (GPT 5.5 - ThinkingDeeper) and edited and summarised afterwards. The author has thoroughly reviewed and edited the AI-generated content mentioned above and takes responsibility for the entire content of the paper.

6.4 Future Plans

Our current results suggest that LLMs can handle basic MSC concepts, but struggle with abstraction, composition, concurrency, and interleaving. These findings motivate future work on formulating and testing concrete hypotheses about recurring LLM “pain patterns” in formal diagram understanding.

6.5 Conclusions

Acknowledgments

References

1. Booch, G., Rumbaugh, J., Jacobson, I.: The Unified Modeling Language user guide. Addison Wesley Longman Publishing Co., Inc., USA (1999)
2. Cheung, K.S., Kaul, M., Jahangirova, G., Mousavi, M.R., Zie, E.: Comparative analysis of carbon footprint in manual vs. llm-assisted code development. In: Proceedings of the 1st International Workshop on Responsible Software Engineering, ResponsibleSE 2025, Trondheim, Norway, June 23-28, 2025. pp. 13–20. ACM (2025). <https://doi.org/10.1145/3711919.3728678>, <https://doi.org/10.1145/3711919.3728678>
3. Elberzhager, F., Gerbershagen, M., Ginkel, J.: Using llms to evaluate architecture documents – results from a digital marketplace environment. In: Dorner, M., Ramler, R., Winkler, D., Bergsmann, J. (eds.) Software Architecture as the Backbone of Software Quality. pp. 65–81. Springer Nature Switzerland, Cham (2026)
4. Hooda, A., Christodorescu, M., Allamanis, M., Wilson, A., Fawaz, K., Jha, S.: Do large code models understand programming concepts? counterfactual analysis for code predicates. In: Proceedings of the 41st International Conference on Machine Learning. ICML’24, JMLR.org (2024)
5. (ITU-T), I.T.S.S.: Recommendation Z.120: Message sequence chart (MSC) (September 1993)
6. Laneve, C., Spanò, A., Ressi, D., Rossi, S., Bugliesi, M.: Understanding code semantics: a benchmark study of llms. International Journal on Software Tools for Technology Transfer (2026). <https://doi.org/10.1007/s10009-026-00842-4>, <https://doi.org/10.1007/s10009-026-00842-4>
7. Ma, W., Liu, S., Lin, Z., Wang, W., Hu, Q., Liu, Y., Zhang, C., Nie, L., Li, L., Liu, Y.: Lms: Understanding code syntax and semantics for code analysis (2024), <https://arxiv.org/abs/2305.12138>
8. Nikiema, S.L., Samhi, J., Kaboré, A.K., Klein, J., Bissyandé, T.F.: The code barrier: What llms actually understand? (2025), <https://arxiv.org/abs/2504.10557>
9. North, M., Atapour-Abarghouei, A., Bencomo, N.: Beyond syntax: How do llms understand code? In: 2025 IEEE/ACM 47th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER). pp. 86–90 (2025). <https://doi.org/10.1109/ICSE-NIER66352.2025.00023>
10. Ouyang, S., Zhang, J.M., Gong, J., Jahangirova, G., Mousavi, M.R., Johns, J., Lee, B.S., Ziolkowski, A., Virginas, B., Noppen, J.: Benchmarking and evaluating vlms for software architecture diagram understanding (2026), <https://arxiv.org/abs/2604.04009>

11. Salles, A., Evers, K., Farisco, M.: Anthropomorphism in ai. *AJOB Neuroscience* **11**(2), 88–95 (2020). <https://doi.org/10.1080/21507740.2020.1740350>, <https://doi.org/10.1080/21507740.2020.1740350>
12. Schmid, L., Hey, T., Armbruster, M., Corallo, S., Fuchß, D., Keim, J., Liu, H., Kozi-
olek, A.: Software architecture meets llms: A systematic literature review (2025), <https://arxiv.org/abs/2505.16697>
13. Soliman, M., Ashraf, E., Abdelsalam, K.M.K., Keim, J., Venkatesh, A.P.S.: Llms
for software architecture knowledge: A comparative analysis among seven llms.
In: Andrikopoulos, V., Pautasso, C., Ali, N., Soldani, J., Xu, X. (eds.) *Software
Architecture*. pp. 99–115. Springer Nature Switzerland, Cham (2026)
14. Soliman, M., Keim, J.: Do large language models contain software architec-
tural knowledge? : An exploratory case study with gpt. In: *2025 IEEE 22nd
International Conference on Software Architecture (ICSA)*. pp. 13–24 (2025).
<https://doi.org/10.1109/ICSA65012.2025.00012>
15. Spiess, C., Devanbu, P., Barr, E.T.: How robustly do llms understand execution
semantics? (2026), <https://arxiv.org/abs/2604.16320>
16. Wei, A., Cao, J., Li, R., Chen, H., Zhang, Y., Wang, Z., Liu, Y., Teix-
eira, T.S.F.X., Yang, D., Wang, K., Aiken, A.: EquiBench: Benchmarking large
language models’ reasoning about program semantics via equivalence check-
ing. In: Christodoulopoulos, C., Chakraborty, T., Rose, C., Peng, V. (eds.)
*Proceedings of the 2025 Conference on Empirical Methods in Natural Lan-
guage Processing*. pp. 33868–33881. Association for Computational Linguistics,
Suzhou, China (Nov 2025). <https://doi.org/10.18653/v1/2025.emnlp-main.1718>,
<https://aclanthology.org/2025.emnlp-main.1718/>
17. Zhou, X., Li, R., Liang, P., Zhang, B., Shahin, M., Li, Z., Yang, C.: Us-
ing llms in generating design rationale for software architecture decisions.
ACM Trans. Softw. Eng. Methodol. (Dec 2025). <https://doi.org/10.1145/3785010>,
<https://doi.org/10.1145/3785010>, just Accepted